

MAST32001 Computational Statistics - Lecture 7

Bootstrap for regression, hypothesis tests, and time series

Klaus Nordhausen

University of Helsinki

WS 2026

Today

1. Bootstrap hypothesis tests
2. Bootstrap for linear regression
3. Residual bootstrap and pairs bootstrap
4. Heteroscedasticity and wild bootstrap
5. Time series bootstrap
6. What can go wrong?

Bootstrap hypothesis tests

Bootstrap tests

Suppose a test statistic

$$T = T(X_1, \dots, X_n)$$

is large when H_0 is false.

A bootstrap test needs a way to simulate data under the null hypothesis.

This is the main difference between bootstrap testing and bootstrap standard errors.

For testing, the resampling mechanism should approximate

$$\mathcal{L}(T \mid H_0).$$

Thus the bootstrap sample must be generated from a version of the data-generating process in which H_0 is true.

Parametric bootstrap test

Algorithm:

1. Fit the model under H_0 .
2. Simulate bootstrap samples from this fitted null model.
3. Recompute the test statistic for each bootstrap sample.
4. Compare the observed statistic to the bootstrap null distribution.

If large values reject,

$$p_{\text{boot}} = \frac{1 + \sum_{b=1}^B I(T^{*(b)} \geq T_{\text{obs}})}{B + 1}.$$

The +1 correction avoids reporting zero due only to finite simulation.

Nonparametric bootstrap test

For a nonparametric bootstrap test we often need to modify the empirical distribution so that the null hypothesis is true.

Example:

$$H_0 : E(X) = \mu_0.$$

If the observed sample has mean $\bar{x} \neq \mu_0$, resampling directly from the empirical distribution estimates the distribution under mean $\bar{x}$, not μ_0 .

A simple recentering is

$$x_i^0 = x_i - \bar{x} + \mu_0.$$

Then bootstrap samples are drawn from $x_1^0, \dots, x_n^0$.

Example: one-sample mean test

```
set.seed(20260812)
x <- rnorm(25, mean = 0.3, sd = 1)
mu0 <- 0
t_obs <- abs((mean(x) - mu0) / (sd(x) / sqrt(length(x))))
x0 <- x - mean(x) + mu0
B <- 9999
t_boot <- replicate(B, {
  xs <- sample(x0, length(x0), replace = TRUE)
  abs((mean(xs) - mu0) / (sd(xs) / sqrt(length(xs))))
})
p_boot <- (1 + sum(t_boot >= t_obs)) / (B + 1)
c(t_obs = t_obs, p_boot = p_boot,
  p_t_test = t.test(x, mu = mu0)$p.value)
```

```
##      t_obs      p_boot  p_t_test
## 0.3416348 0.7345000 0.7355990
```

Bootstrap tests versus permutation tests

Permutation tests often simulate from an exact finite-sample symmetry under H_0 .

Bootstrap tests usually approximate the sampling distribution by simulating from an estimated null-generating process.

Thus:

permutation often emphasizes symmetry,

whereas

bootstrap often emphasizes plug-in estimation.

Both are resampling methods, but their justifications are different.

Linear regression

Linear regression model

Consider

$$Y_i = x_i^\top \beta + \varepsilon_i, \quad i = 1, \dots, n.$$

In matrix form:

$$Y = X\beta + \varepsilon.$$

A standard model assumes

$$E(\varepsilon_i \mid x_i) = 0,$$

and often

$$\varepsilon_i \stackrel{iid}{\sim} (0, \sigma^2).$$

The bootstrap must reflect which part of the data is considered random.

Least-squares estimator

The least-squares estimator is

$$\hat{\beta} = (X^{\top} X)^{-1} X^{\top} Y$$

when $X^{\top} X$ is nonsingular.

The fitted values are

$$\hat{y}_i = x_i^{\top} \hat{\beta}$$

and residuals are

$$\hat{\varepsilon}_i = y_i - \hat{y}_i.$$

Bootstrap methods for regression usually resample either

- ▶ residuals;
- ▶ observation pairs (x_i, y_i) ;
- ▶ model-based errors.

Two main nonparametric regression bootstraps

There are two common variants.

Residual bootstrap

Keep the design matrix X fixed and resample residuals.

Best suited when the design is fixed and errors are approximately iid with constant variance.

Pairs bootstrap

Resample complete observations

$$(x_i, y_i)$$

with replacement.

Best suited when the observations are iid pairs and the design is random.

These answer slightly different modelling questions.

Residual bootstrap

Residual bootstrap idea

The fitted regression model is

$$y_i = \hat{y}_i + \hat{\varepsilon}_i.$$

If the errors are iid, then the residuals approximate the error distribution.

Residual bootstrap algorithm:

1. Fit the regression model.
2. Compute centered residuals

$$\tilde{\varepsilon}_i = \hat{\varepsilon}_i - \bar{\hat{\varepsilon}}.$$

3. Sample residuals with replacement:

$$\varepsilon_i^* \sim \{\tilde{\varepsilon}_1, \dots, \tilde{\varepsilon}_n\}.$$

4. Form

$$y_i^* = \hat{y}_i + \varepsilon_i^*.$$

5. Refit the model to (X, y^*) .

Residual bootstrap assumptions

The residual bootstrap assumes that, after fitting,

$$\varepsilon_1, \dots, \varepsilon_n$$

are approximately iid and independent of the design.

In particular, it assumes approximately constant error variance:

$$\text{Var}(\varepsilon_i \mid x_i) = \sigma^2.$$

If the errors are heteroscedastic, residuals from low- and high-variance regions should not be freely exchanged.

This is a design-respecting but error-exchangeability-based bootstrap.

R: residual bootstrap

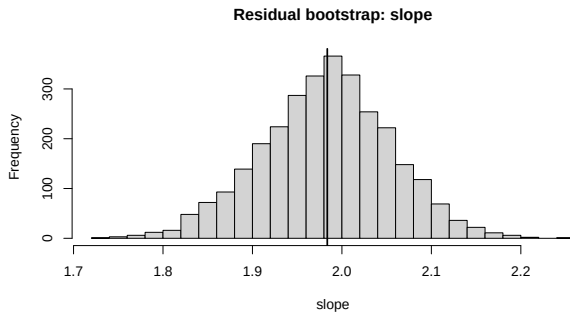
```
set.seed(20260812)
n <- 80
x <- runif(n, 0, 3)
y <- 1 + 2 * x + rnorm(n, sd = 0.6)
fit <- lm(y ~ x)
B <- 3000
coef_resid <- matrix(NA_real_, B, 2)
res <- resid(fit)
res <- res - mean(res)
fitted_y <- fitted(fit)
for (b in seq_len(B)) {
  y_star <- fitted_y + sample(res, n, replace = TRUE)
  coef_resid[b, ] <- coef(lm(y_star ~ x))
}
apply(coef_resid, 2, mean)
```

```
## [1] 0.9091679 1.9825253
```

```
apply(coef_resid, 2, sd)
```

```
## [1] 0.12667016 0.07241331
```

Residual bootstrap distribution



The bootstrap distribution describes the variability of the estimated slope under the fitted regression model with resampled errors.

Pairs bootstrap

Pairs bootstrap idea

If the data consist of iid observations

$$(X_i, Y_i), \quad i = 1, \dots, n,$$

then we may sample complete pairs with replacement.

Algorithm:

1. Draw indices

$$I_1, \dots, I_n$$

with replacement from $\{1, \dots, n\}$.

2. Create

$$(x_j^*, y_j^*) = (x_{I_j}, y_{I_j}).$$

3. Refit the regression model to the bootstrap sample.
4. Repeat.

This treats the design points as random.

Pairs bootstrap assumptions

The pairs bootstrap assumes that the observed pairs are iid draws from a joint distribution.

It can reflect some forms of heteroscedasticity because the relationship between x_i and residual scale is preserved inside the resampled pairs.

But it also changes the design matrix from bootstrap sample to bootstrap sample.

Thus it targets the sampling variability for random-design regression.

R: pairs bootstrap

```
coef_pairs <- matrix(NA_real_, B, 2)
dat <- data.frame(x = x, y = y)
for (b in seq_len(B)) {
  ind <- sample.int(n, n, replace = TRUE)
  dat_star <- dat[ind, ]
  coef_pairs[b, ] <- coef(lm(y ~ x, data = dat_star))
}
apply(coef_pairs, 2, mean)
```

```
## [1] 0.9083624 1.9836763
```

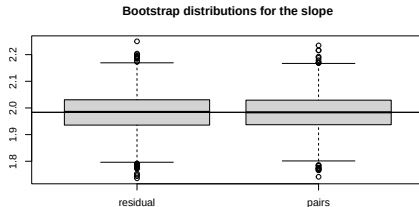
```
apply(coef_pairs, 2, sd)
```

```
## [1] 0.11292889 0.06641776
```

Comparing residual and pairs bootstrap

```
rbind(  
  residual = apply(coef_resid, 2, sd),  
  pairs = apply(coef_pairs, 2, sd)  
)
```

```
##           [,1]      [,2]  
## residual 0.1266702 0.07241331  
## pairs    0.1129289 0.06641776
```



For simple homoscedastic data, the results may be similar.

For heteroscedastic data, they can differ substantially.

Parametric regression bootstrap

If we assume normal homoscedastic errors,

$$\varepsilon_i \sim N(0, \sigma^2),$$

we can simulate

$$y_i^* = \hat{y}_i + \varepsilon_i^*, \quad \varepsilon_i^* \sim N(0, \hat{\sigma}^2).$$

This is parametric because it specifies a model for the error distribution.

It is efficient if the model is correct, but sensitive to model misspecification.

R: parametric regression bootstrap

```
sigma_hat <- summary(fit)$sigma
coef_param <- matrix(NA_real_, B, 2)
for (b in seq_len(B)) {
  y_star <- fitted_y + rnorm(n, sd = sigma_hat)
  coef_param[b, ] <- coef(lm(y_star ~ x))
}

rbind(
  parametric = apply(coef_param, 2, sd),
  residual = apply(coef_resid, 2, sd),
  pairs = apply(coef_pairs, 2, sd)
)
```

```
##           [,1]      [,2]
## parametric 0.1342651 0.07487728
## residual   0.1266702 0.07241331
## pairs      0.1129289 0.06641776
```

Interactive part

R TASK: residual or pairs?

Work with your neighbour.

Use heteroscedastic data:

```
set.seed(20260812)
n <- 100
x <- runif(n, 0, 4)
y <- 1 + 2 * x + rnorm(n, sd = 0.2 + 0.75 * x)
fit <- lm(y ~ x)
```

Tasks:

1. Plot residuals against fitted values.
2. Implement a residual bootstrap for the slope.
3. Implement a pairs bootstrap for the slope.
4. Compare the two bootstrap standard errors.
5. Which one seems more plausible here, and why?

R TASK: one possible solution

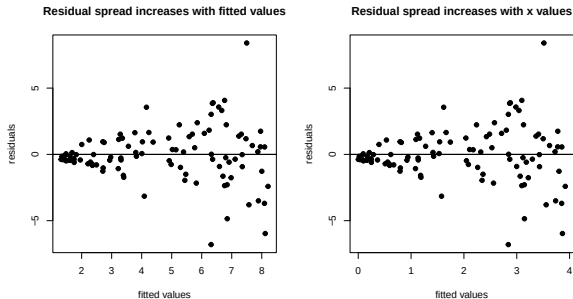
```
set.seed(20260812)
n <- 100
x <- runif(n, 0, 4)
y <- 1 + 2 * x + rnorm(n, sd = 0.2 + 0.75 * x)
fit <- lm(y ~ x)
B <- 2000
slope_resid <- numeric(B)
slope_pairs <- numeric(B)
res <- resid(fit) - mean(resid(fit))
fy <- fitted(fit)
dat <- data.frame(x = x, y = y)
for (b in seq_len(B)) {
  y_star <- fy + sample(res, n, replace = TRUE)
  slope_resid[b] <- coef(lm(y_star ~ x))[2]
  ind <- sample.int(n, n, replace = TRUE)
  slope_pairs[b] <- coef(lm(y ~ x, data = dat[ind, ]))[2]
}
```

R TASK: one possible solution

```
c(  
  residual_boot_se = sd(slope_resid),  
  pairs_boot_se = sd(slope_pairs)  
)
```

```
## residual_boot_se    pairs_boot_se  
##           0.1645472           0.1723904
```

R TASK: diagnostic plot



The residual bootstrap freely moves high-variance residuals to low- x observations and vice versa.

The pairs bootstrap better preserves the observed mean–variance relationship.

Wild bootstrap

Wild bootstrap motivation

For heteroscedastic regression,

$$\text{Var}(\varepsilon_i \mid x_i)$$

may depend on x_i .

The residual bootstrap is then questionable because it treats residuals as exchangeable.

The wild bootstrap keeps the fitted residual attached to its design point but randomly changes its sign or scale:

$$y_i^* = \hat{y}_i + \hat{\varepsilon}_i w_i,$$

where

$$E(w_i) = 0, \quad \text{Var}(w_i) = 1.$$

Wild bootstrap choices

A simple choice is the Rademacher distribution:

$$P(w_i = 1) = P(w_i = -1) = \frac{1}{2}.$$

Then

$$y_i^* = \hat{y}_i \pm \hat{\varepsilon}_i.$$

The residual magnitude remains connected to the design point.

This is useful when heteroscedasticity is present and inference on regression coefficients is desired.

We mention it here as an important extension, but do not study the full theory in this course.

Bootstrap covariance of estimates

Bootstrap covariance matrix

If the bootstrap produces coefficient estimates

$$\hat{\beta}^{*(1)}, \dots, \hat{\beta}^{*(B)} \in \mathbb{R}^p,$$

then the bootstrap covariance estimate is

$$\widehat{\text{Cov}}_{\text{boot}}(\hat{\beta}) = \frac{1}{B-1} \sum_{b=1}^B (\hat{\beta}^{*(b)} - \bar{\beta}^*)(\hat{\beta}^{*(b)} - \bar{\beta}^*)^\top,$$

where

$$\bar{\beta}^* = \frac{1}{B} \sum_{b=1}^B \hat{\beta}^{*(b)}.$$

In R, if rows are bootstrap replicates:

```
cov(coef_pairs)
```

```
##           [,1]      [,2]
## [1,]  0.01275293 -0.005965310
## [2,] -0.00596531  0.004411319
```

Short introduction to time series

What is a time series?

A **time series** is a sequence of observations ordered in time,

$$X_1, X_2, \dots, X_n.$$

Unlike an iid sample, neighbouring observations may be dependent.

Before using tools such as the ACF, we need a notion of **stationarity**.

Strong and weak stationarity

A process (X_t) is **strongly (strictly) stationary** if, for every k , every collection of times $t_1, \dots, t_k$, and every shift h ,

$$(X_{t_1}, \dots, X_{t_k}) \stackrel{d}{=} (X_{t_1+h}, \dots, X_{t_k+h}).$$

Thus the complete joint distribution is unchanged by shifting time.

A process is **weakly (second-order) stationary** if

$$E(X_t) = \mu$$

does not depend on t , and

$$\text{Cov}(X_t, X_{t-h}) = \gamma(h)$$

depends only on the lag h , not on the time t .

If second moments exist, strong stationarity implies weak stationarity when the mean and covariance are finite and time-invariant. The converse is not true in general.

ACF and PACF

For a weakly stationary series, the autocovariance at lag h is

$$\gamma(h) = \text{Cov}(X_t, X_{t-h}),$$

and the autocorrelation function (ACF) is

$$\rho(h) = \frac{\gamma(h)}{\gamma(0)}.$$

The **partial autocorrelation function (PACF)** measures association between X_t and X_{t-h} after accounting for the intermediate lags $1, \dots, h-1$.

The usual interpretation of the ACF as a function of **lag only** relies on stationarity.

AR, MA and ARMA models

An autoregressive model of order p , $AR(p)$, is

$$X_t = c + \phi_1 X_{t-1} + \cdots + \phi_p X_{t-p} + \varepsilon_t.$$

A moving-average model of order q , $MA(q)$, is

$$X_t = \mu + \varepsilon_t + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}.$$

Combining both gives an $ARMA(p, q)$ model:

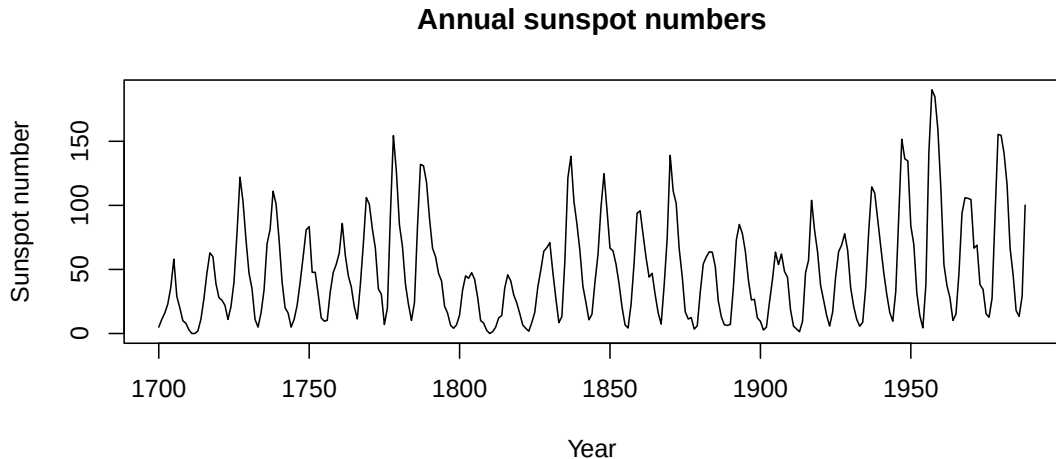
$$X_t = c + \sum_{j=1}^p \phi_j X_{t-j} + \varepsilon_t + \sum_{j=1}^q \theta_j \varepsilon_{t-j}.$$

Here ε_t is white noise: uncorrelated innovations with mean zero and constant variance.

A useful rule of thumb: the PACF of an $AR(p)$ cuts off after lag p , while the ACF of an $MA(q)$ cuts off after lag q . For ARMA models both usually decay gradually.

Real data: annual sunspot numbers

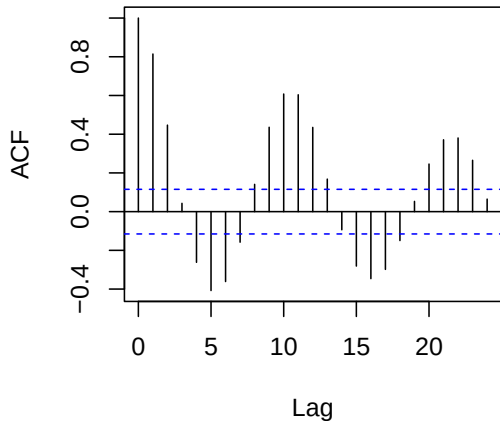
Base R contains the `sunspot.year` time series of annual mean sunspot numbers.



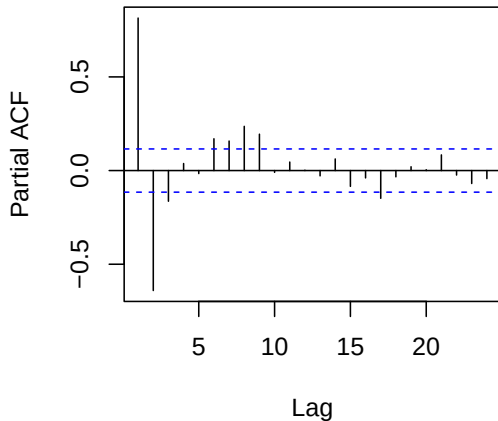
The observations are clearly ordered and show strong temporal structure. Treating these values as iid would discard that structure.

ACF and PACF of the sunspot data

ACF



PACF



The ACF shows how correlation changes with lag. The PACF asks how much additional linear association remains at a lag after the shorter lags have been accounted for.

Time series bootstrap

Why iid bootstrap fails for time series

For time series

$$X_1, \dots, X_n,$$

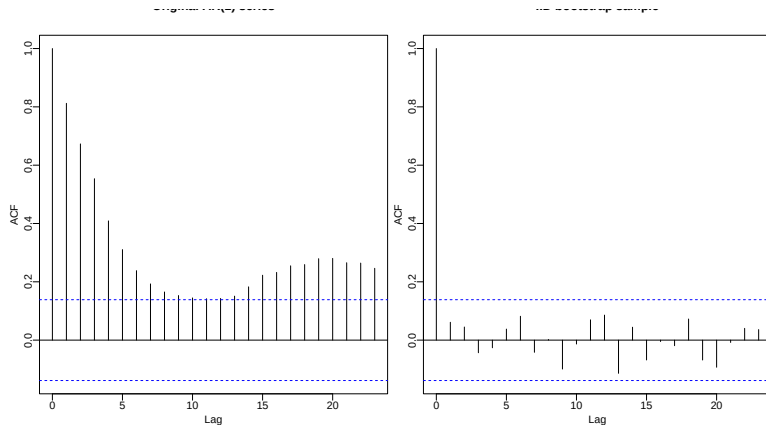
nearby observations are often dependent.

If we sample individual observations with replacement, we destroy

- ▶ autocorrelation;
- ▶ local trends;
- ▶ volatility clustering;
- ▶ seasonal patterns;
- ▶ temporal ordering.

Thus the iid bootstrap usually does not reproduce the structure relevant for inference.

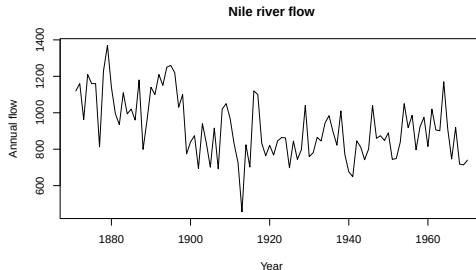
Example: autocorrelation destroyed



The iid bootstrap sample does not preserve serial dependence.

Active task: iid or block bootstrap?

Consider the annual Nile river-flow series available in R.



In pairs, answer **before running any bootstrap**:

1. Would you treat these observations as iid? Why or why not?
2. Inspect the ACF. What does it suggest about dependence?
3. For estimating the uncertainty of the sample mean, would you prefer
 - ▶ an iid bootstrap, or
 - ▶ a moving block bootstrap?
4. What assumption about the series worries you when using an ordinary block bootstrap?

Active task: iid or block bootstrap?

```
acf(Nile)
x <- as.numeric(Nile)
n <- length(x)
B <- 2000
set.seed(20260812)
se_iid <- sd(replicate(
  B, mean(sample(x, n, replace = TRUE))
))
```

Active task: compare the two bootstraps

Now compare the iid bootstrap with a moving block bootstrap using block length $\ell = 8$.

```
moving_block_boot <- function(x, block_length) {  
  n <- length(x)  
  starts <- seq_len(n - block_length + 1)  
  blocks <- lapply(  
    starts,  
    function(i) x[i:(i + block_length - 1)]  
  )  
  k <- ceiling(n / block_length)  
  chosen <- sample(seq_along(blocks), k, replace = TRUE)  
  as.vector(unlist(blocks[chosen]))[seq_len(n)]  
}
```

Active task: compare the two bootstraps

```
set.seed(20260812)
mean_block <- replicate(
  B, mean(moving_block_boot(x, block_length = 8))
)
se_block <- sd(mean_block)
c(iid = se_iid, block = se_block)
```

Discuss:

- ▶ Which standard error is larger?
- ▶ Why does preserving local dependence affect uncertainty about the mean?
- ▶ Does the block bootstrap automatically solve every problem visible in the Nile series?

Active task: solution – dependence

The Nile observations are **not naturally viewed as iid**: they are ordered in time, and the ACF shows serial dependence.

For uncertainty about the sample mean, an iid bootstrap ignores this dependence. A moving block bootstrap is therefore the more appropriate of the two **if a stationary weakly dependent model is reasonable**.

The reason is that

$$\text{Var}(\bar{X})$$

depends not only on the marginal variance, but also on autocovariances between observations.

Thus destroying serial dependence can give the wrong sampling uncertainty.

Active task: solution – iid versus block bootstrap

The code compares

$$\widehat{SE}_{iid} \quad \text{and} \quad \widehat{SE}_{block}.$$

```
x <- as.numeric(Nile)
n <- length(x)
B <- 2000
moving_block_boot_task <- function(x, block_length) {
  n <- length(x)
  starts <- seq_len(n - block_length + 1)
  blocks <- lapply(
    starts,
    function(i) x[i:(i + block_length - 1)])
  )
  k <- ceiling(n / block_length)
  chosen <- sample(seq_along(blocks), k, replace = TRUE)
  as.vector(unlist(blocks[chosen]))[seq_len(n)]
}
```

Active task: solution – iid versus block bootstrap

```
set.seed(20260812)
se_iid <- sd(replicate(
  B, mean(sample(x, n, replace = TRUE))
))

set.seed(20260812)
mean_block <- replicate(
  B, mean(moving_block_boot_task(x, block_length = 8))
)
se_block <- sd(mean_block)
c(iid = se_iid, block = se_block)
```

```
##      iid      block
## 16.72087 30.78532
```

The numerical difference is a consequence of the dependence structure that the two resampling schemes preserve differently.

Active task: solution – but is block bootstrap valid here?

The block bootstrap preserves **local dependence within blocks**, but that is not the only issue.

The Nile plot suggests that the distribution may not be constant over the whole observation period. This raises a stationarity concern.

So the conclusion is not simply

dependent time series $\implies$ block bootstrap.

Instead ask

What structure must the resampling scheme preserve?

For an ordinary moving block bootstrap, stationarity and sufficiently weak dependence are central assumptions.

Can we block-bootstrap any time series?

Not in general. The standard block bootstrap is designed primarily for a **stationary, weakly dependent** process.

Typical asymptotic assumptions include:

- ▶ the process is stationary;
- ▶ dependence weakens as observations become farther apart;
- ▶ suitable moments of the statistic exist;
- ▶ the block length $\ell = \ell_n$ grows with n , but not too quickly:

$$\ell_n \rightarrow \infty, \quad \frac{\ell_n}{n} \rightarrow 0.$$

The exact assumptions depend on the statistic and the theorem being used. Conditions are often expressed using mixing or related weak-dependence assumptions.

So

block bootstrap $\neq$ automatic solution for dependence.

What about non-stationary data?

A standard moving block bootstrap is generally **not justified simply because the data are a time series**.

For example, consider

$$X_t = \beta t + \varepsilon_t.$$

Blocks taken from early and late parts of the series have different distributions. Resampling them as if they were interchangeable does not reproduce the original data-generating mechanism.

Possible strategies depend on the source of non-stationarity:

- ▶ deterministic trend: model/remove the trend and bootstrap suitable residuals;
- ▶ seasonality: preserve or model the seasonal structure;
- ▶ changing variance: use a method designed for that structure;
- ▶ unit roots/integrated processes: ordinary block-bootstrap theory needs modification;
- ▶ structural breaks: resampling across regimes can be inappropriate.

The central question remains:

Which features must the resampling scheme reproduce for the statistic of interest?

Moving block bootstrap

The moving block bootstrap samples consecutive blocks.

For block length ℓ , define blocks

$$B_i = (X_i, \dots, X_{i+\ell-1}), \quad i = 1, \dots, n - \ell + 1.$$

Sample blocks with replacement and concatenate them until a series of length n is obtained.

The idea is to preserve local dependence inside each block.

Moving block bootstrap algorithm

1. Choose a block length ℓ .
2. Form all overlapping blocks

$$B_1, \dots, B_{n-\ell+1}.$$

3. Sample

$$k = \lceil n/\ell \rceil$$

blocks with replacement.

4. Concatenate the sampled blocks.
5. Truncate to length n .
6. Compute the statistic.

The block length controls the bias–variance trade-off.

R: moving block bootstrap – Nile example

We continue with the **Nile river-flow data** from the active task:

$$x = (X_1, \dots, X_n).$$

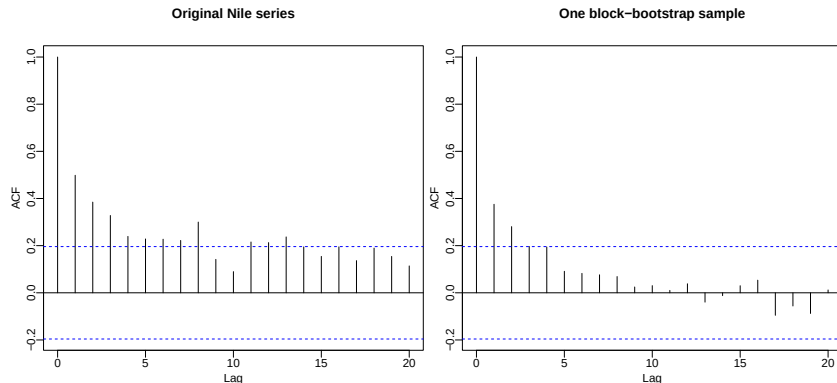
```
x <- as.numeric(Nile)

moving_block_boot <- function(x, l) {
  n <- length(x)
  starts <- seq_len(n - l + 1)
  k <- ceiling(n / l)
  chosen <- sample(starts, size = k, replace = TRUE)
  out <- unlist(lapply(chosen, function(i) x[i:(i + l - 1)]))
  out[seq_len(n)]
}

set.seed(20260812)
xb <- moving_block_boot(x, l = 10)
```

Here xb is **one moving-block bootstrap sample of the Nile series**.

R: original Nile ACF versus one bootstrap ACF



The bootstrap ACF should **not** be expected to equal the original ACF exactly.

The point is that sampling consecutive blocks preserves some of the **local serial dependence**, unlike resampling individual observations independently.

Block bootstrap for the Nile mean

```
set.seed(20260812)
B <- 2000
l <- 10
mean_block <- replicate(B, mean(moving_block_boot(x, l)))
mean_iid <- replicate(B, mean(sample(x, length(x), replace = TRUE)))
c(
  original_mean = mean(x),
  iid_boot_se = sd(mean_iid),
  block_boot_se = sd(mean_block)
)
```

```
## original_mean    iid_boot_se block_boot_se
##      919.35000      16.53081      33.23362
```

For the Nile example, these are two different estimates of the uncertainty of the same sample mean. More generally, with positive serial correlation the iid bootstrap often underestimates uncertainty for the mean.

Choosing block length

Block length ℓ is important.

If ℓ is too small:

- ▶ dependence is not preserved;
- ▶ uncertainty may be underestimated.

If ℓ is too large:

- ▶ few effective blocks are sampled;
- ▶ bootstrap variability may be unstable.

There is no universally best block length.

Practical choices use diagnostics, sensitivity analysis, or data-driven selectors.

Other time series bootstraps

Several variants exist:

- ▶ non-overlapping block bootstrap;
- ▶ moving block bootstrap;
- ▶ circular block bootstrap;
- ▶ stationary bootstrap;
- ▶ sieve bootstrap;
- ▶ residual bootstrap from a fitted time series model.

The right choice depends on the time series structure and the statistic of interest.

Circular block bootstrap

The circular bootstrap treats the series as if

$$X_{n+1} = X_1.$$

Blocks can wrap around the end of the observed series.

This removes edge effects and gives n possible starting positions for blocks of length ℓ .

It is often a convenient modification of the moving block bootstrap.

Model-based time series bootstrap

If an ARMA-type model is plausible, one may:

1. fit the time series model;
2. resample or simulate innovations;
3. generate a new time series from the fitted model;
4. recompute the statistic.

This is parametric or semiparametric depending on how the innovations are generated.

It can work well if the fitted dependence structure is adequate.

When not to bootstrap naively

Examples requiring another method

Naive bootstrap methods are questionable for:

- ▶ dependent data treated as iid;
- ▶ very small samples;
- ▶ extremes and endpoints;
- ▶ heavy tails with infinite variance;
- ▶ post-selection inference;
- ▶ nonregular estimators;
- ▶ boundary parameters;
- ▶ rare events not represented in the data;
- ▶ highly imbalanced categorical data;
- ▶ complex survey designs without design-based resampling.

The correct response is not necessarily “do not bootstrap”.

It is often “do not use this bootstrap scheme.”

Bootstrap after model selection

Suppose we select variables and then bootstrap the selected model as if selection had never happened.

The bootstrap distribution may ignore the additional uncertainty from the selection step.

If the bootstrap includes the full selection procedure inside each resample, results can be unstable and difficult to interpret.

Post-selection inference requires special care.

This is a common example where a simple bootstrap is not automatically valid.

Rare events

If a data set contains no events of a rare type, the empirical distribution assigns that event probability zero.

Then the nonparametric bootstrap cannot generate that event.

This makes naive bootstrap inference unsuitable for estimating very small probabilities beyond the resolution of the data.

A model-based approach, more data, or rare-event simulation may be needed.

Summary

What you should remember

- ▶ Bootstrap hypothesis tests require simulating under the null.
- ▶ In regression, residual bootstrap and pairs bootstrap correspond to different assumptions.
- ▶ Residual bootstrap keeps the design fixed and resamples errors.
- ▶ Pairs bootstrap resamples complete observations and treats the design as random.
- ▶ Parametric regression bootstrap simulates from a fitted error model.
- ▶ Wild bootstrap is useful for heteroscedastic regression.
- ▶ The iid bootstrap is not valid for dependent time series.
- ▶ Block bootstrap methods preserve local dependence.
- ▶ Bootstrap validity depends on matching the resampling scheme to the data-generating process.

Final message

The bootstrap is not one algorithm.

It is a principle:

estimate the data-generating process	→	simulate from it	→	recompute the statistic.
--------------------------------------	---	------------------	---	--------------------------

The difficult part is often the first arrow.

What exactly is being estimated, and which assumptions make that estimate reasonable?